

EXP-2

Student Name: ANSHAJ

UID: 24BAI70503

Branch: AIT-CSE (AIML)

Section/Group: 24AIT_KRG-1A

Semester: 5

Subject Code: 24CSP-310

Subject Name : Advanced Database Management System

- **(A)**

A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

CODE:

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    dept_id INT REFERENCES Departments(dept_id)  
);  
  
CREATE TABLE Subjects (  
    subject_id INT PRIMARY KEY,  
    subject_name VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE Marks (  
    student_id INT REFERENCES Students(student_id),  
    subject_id INT REFERENCES Subjects(subject_id),  
    marks_scored INT NOT NULL,  
    PRIMARY KEY (student_id, subject_id)  
);
```

```
INSERT INTO Departments (dept_id, dept_name) VALUES  
(1, 'Computer Science'),  
(2, 'Mathematics');  
  
INSERT INTO Students (student_id, student_name, dept_id) VALUES  
(101, 'Alex', 1),  
(102, 'Bella', 1),  
(103, 'Charlie', 2);  
  
INSERT INTO Subjects (subject_id, subject_name) VALUES  
(501, 'Data Structures'),  
(502, 'Calculus');  
  
INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES  
(101, 501, 85),  
(101, 502, 90),  
(102, 501, 75),  
(103, 502, 95);
```

OUTPUT SCREENSHOTS:

```

47 SELECT
48     D.DEPT_NAME AS DEPARTMENTS,
49     AVG(M.MARKS_SCORED) AS "AVERAGE MARKS"
50 FROM DEPARTMENTS D
51 JOIN STUDENTS S
52     ON D.DEPT_ID = S.DEPT_ID
53 JOIN MARKS M
54     ON M.STUDENT_ID = S.STUDENT_ID
55 GROUP BY D.DEPT_NAME
56 ORDER BY AVG(M.MARKS_SCORED) DESC;

```

Data Output Messages Notifications

	departments character varying (50)	AVERAGE MARKS numeric
1	Mathematics	95.0000000000000000
2	Computer Science	83.3333333333333333

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Learn how to use the AVG() aggregate function to calculate average marks.
- Apply GROUP BY to organize data department-wise.
- Use ORDER BY to sort the results from highest to lowest average marks.

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to **merge these datasets** and identify **each unique employee** (by EmpID) along with their **lowest recorded salary** across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their **lowest salary**, and the corresponding **Ename**.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
--B
CREATE TABLE A
(
    EmpID INT,
    Ename VARCHAR(50),
    Salary INT
);

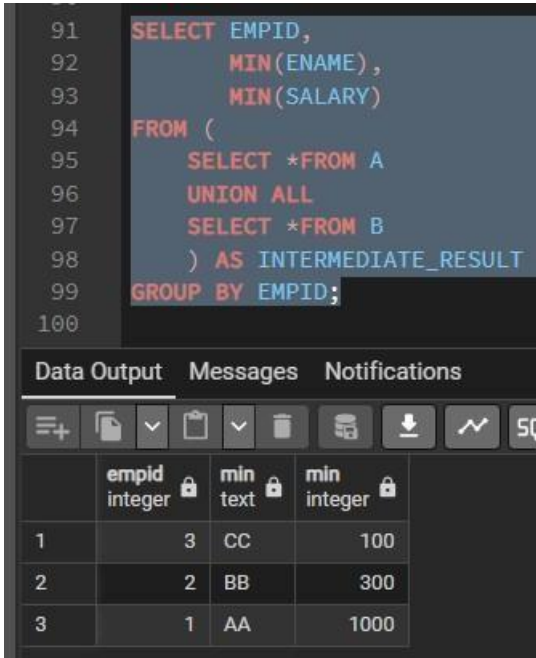
-- Create Table B
CREATE TABLE B
(
    EmpID INT,
    Ename VARCHAR(50),
    Salary INT
);

INSERT INTO A (EmpID, Ename, Salary)
VALUES
(1, 'AA', 1000),
(2, 'BB', 300);

INSERT INTO B (EmpID, Ename, Salary)
VALUES
(2, 'BB', 400),
(3, 'CC', 100);
```

```
SELECT EMPID,  
       MIN(ENAME),  
       MIN(SALARY)  
FROM (  
    SELECT *FROM A  
    UNION ALL  
    SELECT *FROM B  
    ) AS INTERMEDIATE_RESULT  
GROUP BY EMPID;
```

OUTPUT SCREENSHOTS:



```
91 SELECT EMPID,  
92     MIN(ENAME),  
93     MIN(SALARY)  
94 FROM (  
95     SELECT *FROM A  
96     UNION ALL  
97     SELECT *FROM B  
98     ) AS INTERMEDIATE_RESULT  
99 GROUP BY EMPID;  
100
```

	empid integer	min text	min integer
1	3	CC	100
2	2	BB	300
3	1	AA	1000

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use JOIN to combine data from multiple tables.
- Learn how to use the AVG() aggregate function to calculate average marks.
- Apply GROUP BY to organize data department-wise.



- Use ORDER BY to sort the results from highest to lowest average marks.
- Generate a meaningful analytical report by integrating student, department, subject, and marks data.

A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report.

If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the **Department Name, Employee Name, and Salary**, arranged in ascending order of department name.

Note: The employee and department information are maintained in separate normalized tables.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:


```
4
5  --C.
6  CREATE TABLE Department
7  (
8      id INT PRIMARY KEY,
9      dept_name VARCHAR(50)
10 );
11
12 CREATE TABLE Employee
13 (
14     id INT PRIMARY KEY,
15     name VARCHAR(50),
16     salary INT,
17     department_id INT,
18     FOREIGN KEY(department_id)
19     REFERENCES Department(id)
20 );
21
22 INSERT INTO Department
23 VALUES
24 (1,'IT'),
25 (2,'SALES');
26
27 INSERT INTO Employee
28 VALUES
29 (1,'JOE',70000,1),
30 (2,'JIM',90000,1),
31 (3,'HENRY',80000,2),
32 (4,'SAM',60000,2),
33 (5,'MAX',90000,1);
34
```

```
SELECT E.NAME,
       E.SALARY,
       D.DEPT_NAME
FROM
EMPLOYEE AS E
JOIN
DEPARTMENT AS D
ON
E.department_id = D.ID
WHERE E.SALARY IN
(
    SELECT MAX(E1.SALARY)
    FROM EMPLOYEE AS E1
    WHERE E1.department_id = E.department_id
);
```

OUTPUT SCREENSHOTS:

```
135
136 SELECT E.NAME,
137        E.SALARY,
138        D.DEPT_NAME
139 FROM
140 EMPLOYEE AS E
141 JOIN
142 DEPARTMENT AS D
143 ON
144 E.department_id = D.ID
145 WHERE E.SALARY IN
146 (
147     SELECT MAX(E1.SALARY)
148     FROM EMPLOYEE AS E1
149     WHERE E1.department_id = E.department_id
150 )
151
```

Data Output Messages Notifications

	name character varying (50)	salary integer	dept_name character varying (50)
1	JIM	90000	IT
2	HENRY	80000	SALES
3	MAX	90000	IT

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to retrieve data from multiple normalized tables using JOIN operations.
- Learn to identify employees with the highest salary within each department using SQL queries.
- Apply aggregate functions such as MAX() to determine the maximum salary in each department.
- Use subqueries or equivalent SQL techniques to compare employee salaries with the highest departmental salary.
- Handle situations where multiple employees share the highest salary, ensuring all qualifying employees are included in the result.

